

# UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

## Food Store

Sistema de Gestión de Pedidos de Comida

**Materia:** Programación 2

| Integrante       | Comisión   |
|------------------|------------|
| Alfredo Castillo | Comisión 3 |
| Cesia Castillo   | Comisión 5 |
| Rocío Agüero     | Comisión 6 |

**Institución:** Universidad Tecnológica Nacional

**Carrera:** (TUPaD)

**Fecha de entrega:** Junio 2026

# Índice

## 1. Marco Teórico

1.1 Java 21 y Programación Orientada a Objetos

1.2 Colecciones en Java

1.3 Interfaces y Polimorfismo

1.4 Manejo de Excepciones

## 2. Arquitectura y Decisiones Técnicas

2.1 Estructura de paquetes

2.2 Modelo de dominio (UML)

2.3 Capa Repository

2.4 Capa Service

2.5 Capa Menu (UI de consola)

2.6 Flujo de datos y decisiones de diseño

## 3. Capturas del sistema funcionando

## 4. Dificultades y Soluciones

## 5. Bibliografía / Webgrafía

## 6. Links del proyecto

# 1. Marco Teórico

## 1.1 Java 21 y Programación Orientada a Objetos

El proyecto fue desarrollado en **Java 21**, la versión LTS más reciente al momento de la cursada. Java es un lenguaje fuertemente tipado, compilado a bytecode y ejecutado sobre la Máquina Virtual Java (JVM), lo que garantiza portabilidad entre plataformas.

Se aplicaron los cuatro pilares de la **Programación Orientada a Objetos (POO)**:

- **Abstracción:** la clase abstracta Base centraliza los atributos comunes (id, eliminado, createdAt) que comparten todas las entidades del sistema.
- **Encapsulamiento:** todos los atributos son privados y se accede a ellos exclusivamente mediante getters y setters.
- **Herencia:** Categoría, Producto, Usuario, Pedido y DetallePedido extienden de Base, reutilizando su estructura sin duplicación de código.
- **Polimorfismo:** el método abstracto toString() de Base es sobrescrito en cada subclase con una representación específica y útil para el listado en consola.

## 1.2 Colecciones en Java

Dado que la consigna no requiere persistencia en base de datos, toda la información se almacena en memoria durante la ejecución usando el framework de **Colecciones de Java**. Se utilizó **ArrayList<T>** como estructura principal dentro de cada Repository (tablaCategoría, tablaProductos, tablaUsuarios, tablaPedidos). Esta elección permite iteración secuencial, inserción al final en  $O(1)$  amortizado y acceso por índice, suficiente para el volumen de datos esperado en una aplicación de consola educativa.

## 1.3 Interfaces y Polimorfismo

Se definió la interfaz **Calculable** con el método calcularTotal():void. La clase Pedido implementa esta interfaz, delegando en ella la responsabilidad de sumar los subtotales de cada DetallePedido para obtener el total del pedido. Adicionalmente, la interfaz genérica **IRepository<T>** estandariza las operaciones CRUD (guardar, listarTodos, buscarPorId, eliminar) para todas las entidades, aplicando el principio de segregación de interfaces.

## 1.4 Manejo de Excepciones

Se implementaron tres excepciones propias que extienden de **RuntimeException**, lo que las hace unchecked y permite mantener un flujo de código limpio sin cláusulas throws en cascada:

- **EntidadNoEncontradaException:** se lanza cuando se busca un ID que no existe o está marcado como eliminado.
- **StockInvalidoException:** se lanza cuando el precio es negativo o el stock es menor a cero.
- **ValidationException:** se lanza para errores de negocio generales: nombre vacío, mail duplicado, cantidad cero, etc.

Cada menú captura estas excepciones con bloques try-catch específicos, mostrando mensajes claros al operador sin interrumpir la ejecución del programa.

## 2. Arquitectura y Decisiones Técnicas

### 2.1 Estructura de paquetes

El proyecto respeta la arquitectura en tres capas recomendada por la cátedra, organizada bajo el paquete raíz **integrado.prog2**:

| Paquete            | Responsabilidad             | Clases principales                                                                        |
|--------------------|-----------------------------|-------------------------------------------------------------------------------------------|
| <b>entities/</b>   | Modelo de dominio (POO)     | Base, Categoria, Producto, Usuario, Pedido, DetallePedido                                 |
| <b>enums/</b>      | Tipos enumerados            | Rol, EstadoPedido, FormaPago                                                              |
| <b>interfaces/</b> | Contratos de comportamiento | Calculable                                                                                |
| <b>exception/</b>  | Excepciones propias         | EntidadNoEncontradaException, StockInvalidoException, ValidationException                 |
| <b>repository/</b> | Acceso a datos en memoria   | IRepository, CategoriaRepository, ProductoRepository, UsuarioRepository, PedidoRepository |
| <b>service/</b>    | Lógica y reglas de negocio  | CategoriaService, ProductoService, UsuarioService, PedidoService                          |
| <b>menu/</b>       | UI de consola (Scanner)     | MenuCategoria, MenuProducto, MenuUsuario, MenuPedido                                      |
| <b>config/</b>     | Datos iniciales de prueba   | DataInitializer                                                                           |
| <b>(raíz)</b>      | Punto de entrada            | Main.java                                                                                 |

### 2.2 Modelo de dominio (UML)

El modelo de clases sigue fielmente el UML provisto en la consigna. La jerarquía de herencia parte de la clase abstracta **Base**:

| Clase                  | Hereda de         | Atributos propios                                        | Relaciones            |
|------------------------|-------------------|----------------------------------------------------------|-----------------------|
| <b>Base (abstract)</b> | –                 | id: Long eliminado: boolean<br>createdAt: LocalDateTime  | –                     |
| <b>Categoria</b>       | Base              | nombre descripcion                                       | 1:N con Producto      |
| <b>Producto</b>        | Base              | nombre, precio descripcion, stock<br>imagen, disponible  | N:1 con Categoria     |
| <b>Usuario</b>         | Base              | nombre, apellido mail, celular<br>contraseña, rol: Rol   | 1:N con Pedido        |
| <b>Pedido</b>          | Base + Calculable | fecha estado: EstadoPedido total<br>formaPago: FormaPago | 1:N con DetallePedido |

|               |      |                   |                  |
|---------------|------|-------------------|------------------|
| DetallePedido | Base | cantidad subtotal | N:1 con Producto |
|---------------|------|-------------------|------------------|

## 2.3 Capa Repository

Cada entidad tiene su propio Repository que implementa **IRepository<T>**. Internamente se usa un `List<T>` static final (la "tabla en memoria") y un contador `ultimoId` para la auto-generación de IDs. El método **guardar()** actúa como upsert: si el objeto no tiene ID asignado, lo inserta (INSERT); si ya tiene ID, actualiza los campos del objeto existente en la colección (UPDATE). El **soft delete** se implementa marcando `eliminado = true` sin remover el objeto de la colección, garantizando la integridad histórica de los pedidos.

## 2.4 Capa Service

Los servicios concentran todas las reglas de negocio, manteniendo los menús libres de lógica:

- **CategoriaService:** valida nombre no vacío y unicidad de nombre (ignorando mayúsculas/minúsculas). Verifica si la categoría tiene productos asociados antes de eliminar, informando al operador pero permitiendo la baja igualmente (decisión de diseño grupal).
- **ProductoService:** valida nombre no vacío, precio  $\geq 0$  y stock  $\geq 0$  lanzando `StockInvalidoException`. Verifica que la categoría asociada exista y esté activa.
- **UsuarioService:** valida unicidad de mail recorriendo la colección. Al editar, excluye al propio usuario de la validación de unicidad.
- **PedidoService:** valida que el usuario exista, que haya al menos un detalle, que las cantidades sean positivas y que el stock sea suficiente. Acumula el stock total por producto antes de descontarlo, evitando inconsistencias si el mismo producto aparece en múltiples líneas. Implementa rollback manual del stock en caso de excepción durante la creación del pedido.

## 2.5 Capa Menu (UI de consola)

Cada clase de menú recibe un Scanner y los repositorios/servicios necesarios por constructor (inyección de dependencias manual). Los métodos privados `leerEntero()` y `leerLong()` capturan `NumberFormatException` en un loop hasta recibir entrada válida, garantizando que el programa nunca falle por una entrada no numérica.

## 2.6 Flujo de datos y decisiones de diseño

Decisiones técnicas relevantes tomadas durante el desarrollo:

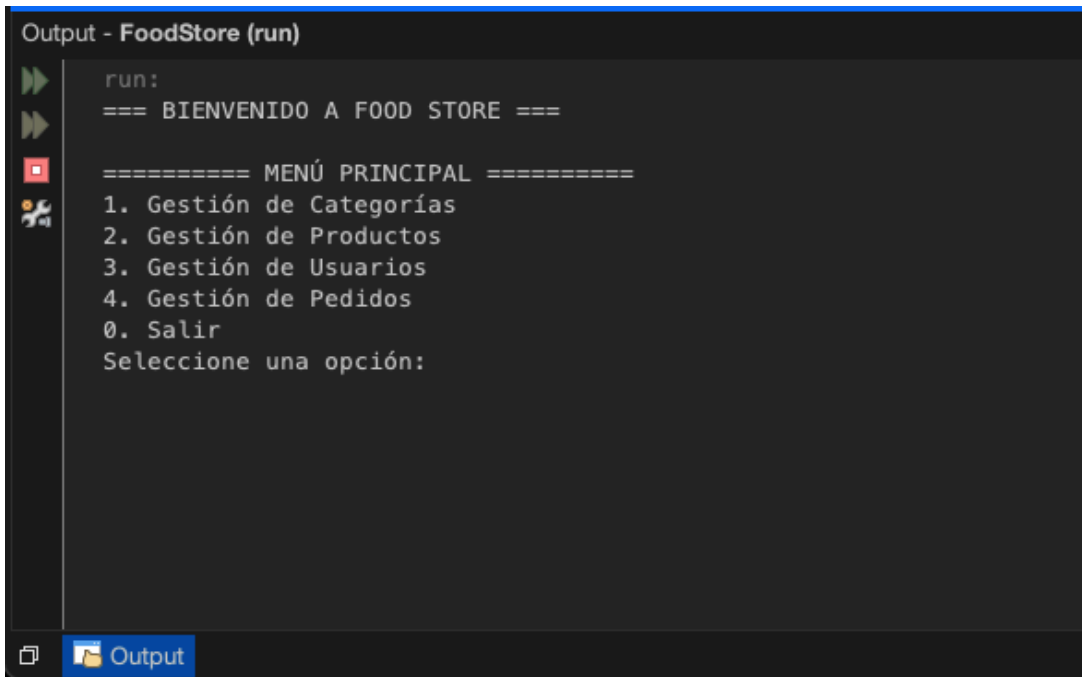
- **IDs autogenerados:** se optó por un contador estático `Long` en cada Repository en lugar de `UUID`, para facilitar la selección por ID desde el menú de consola.
- **Rollback de stock:** en `PedidoService.crearPedido()`, si ocurre una excepción durante la carga de detalles, se revierten manualmente las reducciones de stock aplicadas hasta ese punto, preservando la consistencia de los datos en memoria.
- **DataInitializer:** se precargan datos de prueba (1 categoría, 4 usuarios) al inicio del programa para facilitar la demostración sin necesidad de cargar todo desde cero en cada ejecución.

- **Encapsulamiento del Scanner:** el Scanner se instancia una sola vez en Main y se pasa a todos los menús. Se cierra al salir del programa para evitar resource leaks.
- **Switch expressions (Java 14+):** se usó la sintaxis moderna `case X -> accion()` en los switch del menú principal, aprovechando las características de Java 21.

### 3. Capturas del sistema funcionando

A continuación se muestran las pantallas representativas del sistema. Las capturas corresponden a una ejecución real del programa compilado con Java 21 desde NetBeans IDE.

#### Pantalla 1 – Menú Principal



```
Output - FoodStore (run)

run:
=== BIENVENIDO A FOOD STORE ===

===== MENÚ PRINCIPAL =====
1. Gestión de Categorías
2. Gestión de Productos
3. Gestión de Usuarios
4. Gestión de Pedidos
0. Salir
Seleccione una opción:
```

## Pantalla 2 – Gestión de Categorías (Listar + Crear)

```
Output - FoodStore (run)

===== MENÚ PRINCIPAL =====
1. Gestión de Categorías
2. Gestión de Productos
3. Gestión de Usuarios
4. Gestión de Pedidos
0. Salir
Seleccione una opción: 1

--- GESTIÓN DE CATEGORÍAS ---
1. Listar categorías
2. Crear categoría
3. Editar categoría
4. Eliminar categoría
0. Volver al menú principal
Seleccione una opción: 1
-> Categoría [ID: 1 | Nombre: Comidas Rápidas | Descripción: Hamburguesas, pizzas y minutas | Creada: 2026-06-21T16:13:20.170945 | Eliminada: No]
```

## Pantalla 3 – Creación de Pedido con Detalles

```
--- GESTIÓN DE PEDIDOS ---
1. Listar pedidos
2. Crear pedido
3. Editar pedido (Estado/Forma Pago)
4. Eliminar pedido
0. Volver al menú principal
Seleccione una opción: 2
ID del usuario que realiza la compra: 2
Debe seleccionar una forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Opción: 1
¿Desea ver el catálogo de productos disponibles? (S/N): s

--- CATÁLOGO DE PRODUCTOS ---
-> Producto [ID: 1 | Nombre: Hamburguesa | Precio: $1500,00 | Stock: 5 | Categoría: Comidas Rápidas | Disponible: Sí | Eliminado: No]

--- CARGA DE DETALLES ---
Debe ingresar el ID del Producto: 1
Debe ingresar la Cantidad: 2
¿Desea agregar algun otro producto al pedido? (S/N): n
Pedido creado!. ID generado: 1 | Total final: $3000.0
```

## Pantalla 4 – Manejo de errores y validaciones

```
--- GESTIÓN DE PRODUCTOS ---
1. Listar productos
2. Crear producto
3. Editar producto
4. Eliminar producto
0. Volver al menú principal
Seleccione una opción: 2
Nombre: Pizza
Descripción: Pizza de anchoas
Precio: -500
Stock: 5
Imagen (URL o nombre de archivo): pizza.png
¿Disponible? (S/N): s
ID de la categoría: 1
Error al crear el producto: El precio no puede ser negativo
```

```
--- GESTIÓN DE USUARIOS ---
1. Listar usuarios
2. Crear usuario
3. Editar usuario
4. Eliminar usuario
0. Volver al menú principal
Seleccione una opción: 2
Nombre: alex
Apellido: belisario
Mail: usuario@foodstore.com
Celular: 11556677
Contraseña: 1234
Rol: 1) ADMIN 2) USUARIO
Seleccione una opción: 2
Error al crear el usuario: Ya existe un usuario registrado con el mail usuario@foodstore.com
```

## 4. Dificultades y Soluciones

### 1. Rollback manual del stock en pedidos

**Problema:** Al crear un pedido con múltiples productos, si ocurría una excepción en el medio de la carga de detalles (por ejemplo, al agregar el tercer producto de cuatro), el stock de los productos anteriores ya había sido descontado, dejando los datos en estado inconsistente.

**Solución:** Se implementó un mecanismo de rollback manual en PedidoService: se mantienen dos listas auxiliares (productosModificados y cantidadesRestadas) que registran cada descuento aplicado. En el bloque catch, se itera sobre esas listas sumando de vuelta el stock a cada producto afectado antes de propagar la excepción.



## 2. Validación de unicidad al editar

**Problema:** Al editar un usuario o categoría, la validación de unicidad (mail o nombre) rechazaba el propio registro porque encontraba su valor actual en la colección, impidiendo guardar sin cambios.

**Solución:** Se agregó un parámetro `idAEditar` en los métodos de validación privados. Antes de comparar, se verifica si el item encontrado en la colección es el mismo que se está editando (usando `getId().equals(idAEditar)`), y en ese caso se lo excluye de la validación.

## 3. Coordinación de la interfaz IRepository

**Problema:** Al definir la interfaz genérica `IRepository`, surgieron dudas sobre cómo los métodos específicos (como `buscarPorMail` en `Usuario`) debían integrarse sin romper el contrato genérico.

**Solución:** Se decidió que `IRepository` solo define las operaciones CRUD básicas comunes a todas las entidades. Las consultas específicas (mail, categoría) se implementan como métodos adicionales en el `Repository` concreto correspondiente, sin afectar la interfaz genérica.

## 4. Soft delete e integridad referencial en memoria

**Problema:** Al eliminar lógicamente una categoría, los productos que referenciaban esa categoría quedaban con una referencia a un objeto marcado como eliminado. Del mismo modo, al eliminar usuarios, los pedidos históricos perdían la referencia a un usuario activo.

**Solución:** Se optó por avisar al operador cuando una categoría tiene productos asociados (sin bloquear la baja), porque la consigna lo permite. Para usuarios, se garantiza que el soft delete solo afecta la visibilidad en listados pero los pedidos históricos siguen accesibles y muestran los datos del usuario aunque esté marcado como eliminado, ya que el objeto persiste en memoria.

## 5. Bibliografía / Webgrafía

- [1] **Oracle**. Java SE 21 Documentation. Disponible en: <https://docs.oracle.com/en/java/javase/21/>
- [2] **Oracle**. The Java Tutorials – Collections. Disponible en: <https://docs.oracle.com/javase/tutorial/collections/>
- [3] **Oracle**. The Java Tutorials – Exceptions. Disponible en: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- [4] **Baeldung**. Guide to Java 21 Switch Expressions. Disponible en: <https://www.baeldung.com/java-switch-expression>
- [5] **Baeldung**. Java Generics and Wildcards. Disponible en: <https://www.baeldung.com/java-generics>
- [6] **Cátedra UTN TUPaD**. Material teórico de Programación 2 – POO, Colecciones e Interfaces. Disponible en: Campus virtual UTN (Aula virtual de la materia)
- [7] **Cátedra UTN TUPaD**. Consigna del Trabajo Práctico Integrador – Programación 2 (2026). Disponible en: Documento PDF provisto por la cátedra

## 6. Links del Proyecto

A continuación se listan los enlaces requeridos por la consigna. Todos cuentan con permisos públicos de visualización.

| Recurso                               | URL                                                                                                                         |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Repositorio GitHub                    | <a href="https://github.com/CesiaCastilla/UTN-TPI-Programacion2">https://github.com/CesiaCastilla/UTN-TPI-Programacion2</a> |
| Documentación PDF<br>(este documento) | Incluido en la raíz del repositorio GitHub                                                                                  |
| Video demostración<br>(10–15 min)     | <a href="https://[COMPLETAR-CON-LINK-VIDEO]">https://[COMPLETAR-CON-LINK-VIDEO]</a>                                         |

Nota: reemplazar [COMPLETAR-CON-LINK-VIDEO] con las URLs definitivas antes de la entrega.